

✓ GUION DE CLASE - Manejo de archivos en Python

Fuente: <https://www.freecodecamp.org/espanol/news/python-como-escribir-en-un-archivo-abrir-leer-escribir-y-otras-funciones-de-archivos-explicadas/>

Aprender a abrir, leer, escribir, modificar y eliminar archivos en Python.
Además, aplicar esto con la micro:bit para registrar y guardar datos.

ESTRUCTURA GENERAL

Introducción: Qué es un archivo y por qué es importante.

Parte teórica: Modos, lectura, escritura, contextos, errores.

Ejemplo práctico: Crear, leer y modificar un archivo.

Micro:bit + Python: Guardar datos del sensor

Ejercicios y cierre

INTRODUCCIÓN

Hola a todos, hoy aprenderemos a manejar archivos en Python. Esto nos permite guardar información, leer datos guardados o modificar documentos desde un programa. Es una parte clave de la programación, porque casi todos los programas reales guardan datos.

Qué aprenderán: Cómo abrir y crear archivos.

Cómo leer y escribir información. Qué significan los modos de apertura.

Cómo evitar errores comunes.

Y al final, cómo usar esto con la micro:bit para guardar datos de sensores.

PARTE TEÓRICA

¿Qué es un archivo?

Un archivo es un conjunto de datos guardado en la memoria del computador (por ejemplo, .txt, .csv, .py, etc.).

Python usa la función `open()` para acceder a ellos.

Cómo abrir un archivo

La forma básica es:

```
f = open("archivo.txt", "r")
```

El primer parámetro es el nombre o ruta del archivo. El segundo parámetro es el modo.

✓ Modos principales de apertura

Modo	Qué hace	Si el archivo no existe
"r"	Leer	Error
"w"	Escribir (borra todo lo anterior)	Lo crea
"a"	Agregar al final	Lo crea
"x"	Crear uno nuevo	Error si ya existe
"t"	Modo texto (por defecto)	—
"b"	Modo binario (imágenes, videos)	—

Ejemplo:

```
f = open("nombres.txt", "w")
f.write("Hola mundo\n")
f.close()
```

Esto crea el archivo si no existe, escribe "Hola mundo" y luego lo cierra.

Leer un archivo

Para leer, se usa el modo "r" y los métodos del objeto archivo:

```
f = open("nombres.txt", "r")
print(f.read())    #lee todo
f.close()
```

Otros métodos: `read(n)` → lee n caracteres. `readline()` → lee una línea.

`readlines()` → devuelve una lista de todas las líneas.

Ejemplo:

```
f = open("nombres.txt", "r")
lineas = f.readlines()
print(lineas)
f.close()
```

Salida:

```
['Ana\n', 'Luis\n', 'Marta\n']
```

Agregar texto a un archivo existente

Si quieres escribir sin borrar el contenido anterior:

```
f = open("nombres.txt", "a")
f.write("Carlos\n")
f.close()
```

Esto agrega “Carlos” al final del archivo.

Mejor práctica: usar “with”

En lugar de tener que cerrar el archivo manualmente, se puede usar:

```
with open("nombres.txt","r") as f:
    print(f.read())
```

El archivo se cierra automáticamente al final del bloque with.

Eliminar un archivo

```
import os
os.remove("nombres.txt")
```

Esto elimina el archivo permanentemente.

Manejo de errores

A veces el archivo no existe o no se puede abrir. Usamos try y except para manejar eso:

```
try:  
    f = open("no_existe.txt")  
except FileNotFoundError:  
    print("El archivo no existe")
```

También puede aparecer:

PermissionError → no tienes permiso.

IsADirectoryError → intentas abrir una carpeta como archivo.

PARTE PRÁCTICA

Demostración paso a paso:

1. Crear un archivo llamado datos.txt.
2. Escribir tres líneas.
3. Leerlas con read() y readlines().
4. Agregar una nueva línea al final.
5. Probar abrir un archivo inexistente y manejar el error.
6. Usar "with" para leer de nuevo sin necesidad de close().

Código ejemplo:

```
# 1. Escribir
with open("datos.txt", "w") as f:
    f.write("Ana\nLuis\nMarta\n")

# 2. Leer
with open("datos.txt", "r") as f:
    print(f.read())

# 3. Agregar
with open("datos.txt", "a") as f:
    f.write("Carlos\n")

# 4. Leer con manejo de errores
try:
    with open("no_existe.txt") as f:
        print(f.read())
except FileNotFoundError:
    print("El archivo no se encontró.")
```

MICRO:BIT

✓ EJERCICIOS

1. Crear y escribir:

Crea un archivo llamado saludo.txt y escribe dentro la frase: "Hola, este es mi primer archivo en Python."

2. Leer archivo completo:

Abre el archivo saludo.txt y muestra todo su contenido en pantalla.

3. Leer línea por línea:

Crea un archivo frutas.txt con varias frutas (una por línea). Luego, escribe un programa que lea cada línea y la muestre numerada, así:

1. Manzana

2. Pera

3. Naranja

4. Agregar texto:

Agrega al archivo frutas.txt una nueva fruta al final sin borrar las anteriores.

5. Contar líneas: Cuenta cuántas líneas tiene el archivo frutas.txt y muestra el resultado.

1 Comienza a programar o generar con IA.

